

```

1 import torch
2 import torch.nn as nn
3 import numpy as np
4 import math
5 import copy
6 from tqdm.auto import tqdm
7 from typing import Any, Dict, Optional, Callable, Sequence
8 import pandas as pd
9 import matplotlib.pyplot as plt
10 import torch
11 import torch.nn as nn
12 from torch.utils.data import DataLoader, Dataset
13 from typing import Optional, Tuple, Union, Literal, List
14

```

Functions

Utils

```

1 sigmoid = nn.Sigmoid()
2 softmax = nn.Softmax(dim=1)
3
4 def get_device() -> torch.device:
5     return torch.device("cuda") if torch.cuda.is_available() else torch.device("cpu")
6
7 get_device()

```

```
device(type='cpu')
```

Dataset functions

Split data

```

1 from sklearn.model_selection import train_test_split
2
3 def random_split_xy(
4     X,
5     y,
6     val_frac: float = 0.1,
7     test_frac: float = 0.2,
8     seed: int = 0
9 ):
10     """
11     Randomly split (X, y) into train/val/test using sklearn's train_test_split.
12
13     Inputs:
14         X: array-like, shape (n, d_x) (or any indexable array-like)
15         y: array-like, shape (n, d_y) or (n,) or (n,1) or one-hot (n,C)
16         val_frac: fraction of full data to use for validation
17         test_frac: fraction of full data to use for test
18         seed: random_state
19
20     Returns:
21         X_train, y_train, X_val, y_val, X_test, y_test
22     """
23
24     # 1) Split out test
25     X_tmp, X_test, y_tmp, y_test = train_test_split(
26         X,
27         y,
28         test_size=test_frac,
29         random_state=seed,

```

```

30     shuffle=True
31 )
32
33 # 2) Split tmp into train/val
34 X_train, X_val, y_train, y_val = train_test_split(
35     X_tmp,
36     y_tmp,
37     test_size=val_frac/(1-test_frac),
38     random_state=seed,
39     shuffle=True
40 )
41
42 return (
43     X_train, y_train,
44     X_val, y_val,
45     X_test, y_test
46 )

```

Dataset

```

1 from torch.utils.data import Dataset, DataLoader
2
3 class TabularDataset(Dataset):
4     def __init__(self, X, y):
5         self.X = X
6         self.y = y
7
8     def __len__(self):
9         return len(self.X)
10
11     def __getitem__(self, idx):
12         return self.X[idx], self.y[idx]

```

Models

```

1 class LinearModel(nn.Module):
2     """
3     Linear model (possibly multi-output).
4     Input:  x: (B, d_in)
5     Output: y_hat: (B, d_out)
6     """
7     def __init__(self,
8                 d_in: int,
9                 d_out: int = 1,
10                 bias: bool = True):
11         super().__init__()
12         self.linear = nn.Linear(d_in, d_out, bias=bias)
13
14     def forward(self, x: torch.Tensor) -> torch.Tensor:
15         return self.linear(x)
16
17 class MLP(nn.Module):
18     """
19     MLP with user-specified hidden layer sizes.
20
21     Input:  x: (B, d_in)
22     Output: y_hat: (B, d_out)
23
24     Example:
25     model = MLP(d_in=10, d_hidden=[128, 64, 32], d_out=1)
26     """
27     def __init__(self,
28                 d_in: int,
29                 d_hidden: Sequence[int],
30                 d_out: int = 1,
31                 activation: str = "relu",
32                 dropout: float = 0.0,
33                 bias: bool = True):

```

```

34     super().__init__()
35
36     act = {
37         "relu": nn.ReLU(),
38         "tanh": nn.Tanh(),
39         "sigmoid": nn.Sigmoid(),
40     }.get(activation)
41
42     if act is None:
43         raise ValueError(f"Unknown activation: {activation}")
44
45     dims = [d_in] + list(d_hidden) + [d_out]
46
47     layers = []
48     for i in range(len(dims) - 1):
49         layers.append(nn.Linear(dims[i], dims[i + 1], bias=bias))
50         if i < len(dims) - 2: # not the last layer
51             layers.append(act)
52             if dropout > 0.0:
53                 layers.append(nn.Dropout(dropout))
54
55     self.net = nn.Sequential(*layers)
56
57     def forward(self, x: torch.Tensor) -> torch.Tensor:
58         return self.net(x)
59
60 import torch.nn.functional as F
61
62 class MF(torch.nn.Module):
63     def __init__(self, n, m, rank):
64         super(MF, self).__init__()
65         self.U = torch.nn.Embedding(n, rank)
66         self.V = torch.nn.Embedding(m, rank)
67
68         with torch.no_grad():
69             self.U.weight.copy_(F.normalize(self.U.weight, p=2, dim=1))
70             self.V.weight.copy_(F.normalize(self.V.weight, p=2, dim=1))
71
72     def forward(self, x):
73         i = x[:,0]
74         j = x[:,1]
75         z = self.U(i.squeeze()) * self.V(j.squeeze())
76         z = torch.sum(z, dim=1)
77         return z[:,None]

```

✓ Training functions

Adam

```

1 from torch import optim
2
3 def fit_adam(*,
4     model: nn.Module,
5     train_data: Dataset,
6     val_data: Dataset,
7     batch_size: int,
8     early_stop_patience: int,
9     early_stop_tol: float = 0.0,
10    eval_every: int = 100,
11    lr: float = 1e-3,
12    weight_decay: float = 0.0,
13    n_steps: int = 10_000,
14    loss_fn: Optional[Callable[[torch.Tensor, torch.Tensor], torch.Tensor]] = None,
15    device: Optional[torch.device] = None,
16    verbose: bool = True
17 ) -> Dict[str, Any]:
18
19     train_dataloader = DataLoader(train_data, batch_size=batch_size, shuffle=True)

```

```

20 val_dataloader = DataLoader(val_data, batch_size=batch_size, shuffle=False)
21
22 dev = device if device is not None else next(model.parameters()).device
23 model.to(dev)
24 opt = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
25
26 history = []
27 best_val = float("inf")
28 best_state = copy.deepcopy(model.state_dict())
29 bad_evals = 0
30
31 last_train_loss = math.nan
32 last_val_loss = math.nan
33
34 @torch.no_grad()
35 def eval_val():
36     model.eval()
37     tot, n = 0.0, 0
38     for x, y in val_dataloader:
39         x = x.to(dev)
40         y = y.to(dev)
41         pred = model(x)
42         if loss_fn is None:
43             raise ValueError("loss_fn must be provided when dataloaders yield (x, y).")
44         loss = loss_fn(pred, y)
45         bs = x.shape[0]
46         tot += float(loss) * bs
47         n += bs
48     return tot / max(n, 1)
49
50 step = 0
51 epoch = 0
52 stop = False
53
54 pbar = tqdm(total=n_steps, disable=not verbose, desc="train", unit="step")
55 try:
56     while (not stop) and (step < n_steps):
57         pbar.set_description(f"epoch {epoch}")
58
59         for x, y in train_dataloader:
60             step += 1
61
62             model.train()
63             opt.zero_grad(set_to_none=True)
64
65             x = x.to(dev)
66             y = y.to(dev)
67
68             pred = model(x)
69             if loss_fn is None:
70                 raise ValueError("loss_fn must be provided when dataloaders yield (x, y).")
71
72             loss = loss_fn(pred, y)
73             loss.backward()
74             opt.step()
75
76             last_train_loss = float(loss.detach().cpu())
77
78             do_eval = (step == 1) or (step % eval_every == 0) or (step == n_steps)
79             if do_eval:
80                 last_val_loss = float(eval_val())
81                 history.append({
82                     "step": step,
83                     "train_loss": last_train_loss,
84                     "val_loss": last_val_loss,
85                 })
86
87                 if last_val_loss < best_val - early_stop_tol:
88                     best_val = last_val_loss
89                     best_state = copy.deepcopy(model.state_dict())
90                     bad_evals = 0
91             else:

```

```

92         bad_evals += 1
93         if bad_evals >= early_stop_patience:
94             stop = True
95     else:
96         history.append({
97             "step": step,
98             "train_loss": last_train_loss,
99             "val_loss": math.nan,
100         })
101
102     # show latest train loss every step; val updates on eval steps (otherwise stays last value)
103     pbar.set_postfix(
104         train=f"{last_train_loss:.4g}" if math.isfinite(last_train_loss) else "nan",
105         val=f"{last_val_loss:.4g}" if math.isfinite(last_val_loss) else "nan",
106         best_val=f"{best_val:.4g}" if math.isfinite(best_val) else "inf",
107         bad=bad_evals
108     )
109     pbar.update(1)
110
111     if stop or (step >= n_steps):
112         stop = True
113         break
114
115     epoch += 1
116 finally:
117     pbar.close()
118
119 model.load_state_dict(best_state)
120 return {"model": model, "history": history, "best_val_loss": best_val}

```

LBFGS

```

1 def fit_lbfgs(*,
2     model: nn.Module,
3     train_data: Dataset,
4     val_data: Dataset,
5     loss_fn: Callable[[torch.Tensor, torch.Tensor], torch.Tensor],
6     lr: float = 1.0,
7     l2: float = 0.0,
8     max_iter: int = 200,
9     device: Optional[torch.device] = None
10 ) -> Dict[str, Any]:
11     """
12     Full-batch L-BFGS with Strong Wolfe line search.
13     - Runs the optimizer "normally" (single opt.step; iterations handled internally by LBFGS).
14     - Manual L2: objective = loss_fn(...) + l2 * sum(||p||^2).
15     - Evaluates val once at the end.
16     - Promotes model + float tensors to float64 (recommended for LBFGS).
17     """
18
19     def _to_tensor(a):
20         if torch.is_tensor(a):
21             return a
22         return torch.as_tensor(a)
23
24     def _get_full_xy(ds: Dataset):
25         if hasattr(ds, "X") and hasattr(ds, "y"):
26             return _to_tensor(getattr(ds, "X")), _to_tensor(getattr(ds, "y"))
27
28     x0, y0 = ds[0]
29     x0t, y0t = _to_tensor(x0), _to_tensor(y0)
30     if hasattr(x0t, "shape") and len(getattr(x0t, "shape", ())) > 0 and x0t.shape[0] == len(ds):
31         return x0t, y0t
32
33     xs, ys = [], []
34     for i in range(len(ds)):
35         xi, yi = ds[i]
36         xs.append(_to_tensor(xi))
37         ys.append(_to_tensor(yi))
38     return torch.stack(xs, dim=0), torch.stack(ys, dim=0)

```

```

39
40 dev = device if device is not None else next(model.parameters()).device
41 model.to(dev)
42 model.double()
43
44 x_tr, y_tr = _get_full_xy(train_data)
45 x_va, y_va = _get_full_xy(val_data)
46
47 # Only convert to double if the tensor is floating point. Indices should remain long.
48 x_tr = x_tr.to(dev)
49 if x_tr.is_floating_point():
50     x_tr = x_tr.to(dtype=torch.double)
51
52 y_tr = y_tr.to(dev)
53 if y_tr.is_floating_point():
54     y_tr = y_tr.to(dtype=torch.double)
55
56 x_va = x_va.to(dev)
57 if x_va.is_floating_point():
58     x_va = x_va.to(dtype=torch.double)
59
60 y_va = y_va.to(dev)
61 if y_va.is_floating_point():
62     y_va = y_va.to(dtype=torch.double)
63
64 def _l2_penalty() -> torch.Tensor:
65     if l2 == 0.0:
66         return torch.zeros(), device=dev, dtype=torch.double)
67     s = torch.zeros(), device=dev, dtype=torch.double)
68     for p in model.parameters():
69         if p.requires_grad:
70             s = s + p.pow(2).sum()
71     return l2 * s
72
73 opt = optim.LBFGS(
74     model.parameters(),
75     lr=lr,
76     max_iter=max_iter,
77     line_search_fn="strong_wolfe"
78 )
79
80 def closure():
81     opt.zero_grad(set_to_none=True)
82     model.train()
83     pred = model(x_tr)
84     data_loss = loss_fn(pred, y_tr)
85     obj = data_loss + _l2_penalty()
86     obj.backward()
87     return obj
88
89 # Run LBFGS "normally": max_iter is handled inside this call.
90 opt.step(closure)
91
92 @torch.no_grad()
93 def eval_once(x, y):
94     model.eval()
95     pred = model(x)
96     data_loss = loss_fn(pred, y)
97     obj = data_loss + _l2_penalty()
98     return float(data_loss.detach().cpu()), float(obj.detach().cpu())
99
100 train_loss, train_obj = eval_once(x_tr, y_tr)
101 val_loss, val_obj = eval_once(x_va, y_va)
102
103 return {
104     "model": model,
105     "train_loss": train_loss, # data loss only
106     "train_obj": train_obj, # data loss + manual L2
107     "val_loss": val_loss, # data loss only
108     "val_obj": val_obj # data loss + manual L2
109 }

```

Prediction

```

1 @torch.no_grad()
2 def predict(*,
3   model: nn.Module,
4   data: Dataset,
5   batch_size: int = 128,
6   device: Optional[torch.device] = None,
7   return_targets: bool = True,
8   to_numpy: bool = False
9 ) -> Union[torch.Tensor, Tuple[torch.Tensor, torch.Tensor]]:
10  """
11  Runs model on `data` and returns predictions (optionally with targets).
12
13  Assumes each batch yields either:
14  - x
15  - (x, y)
16  """
17  dev = device if device is not None else next(model.parameters()).device
18  model.to(dev)
19  model.eval()
20
21  loader = DataLoader(data, batch_size=batch_size, shuffle=False)
22
23  preds = []
24  targets = []
25
26  for batch in loader:
27      if isinstance(batch, (tuple, list)) and len(batch) == 2:
28          x, y = batch
29      else:
30          x, y = batch, None
31
32      x = x.to(dev)
33      y_hat = model(x)
34
35      preds.append(y_hat.detach().cpu())
36
37      if return_targets:
38          if y is None:
39              raise ValueError("return_targets=True but dataset/dataloader did not yield (x, y).")
40          targets.append(y.detach().cpu())
41
42  y_hat_all = torch.cat(preds, dim=0) if len(preds) > 0 else torch.empty(0)
43
44  if not return_targets:
45      return y_hat_all.numpy() if to_numpy else y_hat_all
46
47  y_all = torch.cat(targets, dim=0) if len(targets) > 0 else torch.empty(0)
48  if to_numpy:
49      return y_hat_all.numpy(), y_all.numpy()
50
51  return y_hat_all, y_all

```

Eval functions

```

1 from sklearn.metrics import (
2   accuracy_score,
3   precision_recall_fscore_support,
4   log_loss,
5   roc_auc_score,
6   mean_squared_error,
7   mean_absolute_error,
8   brier_score_loss,
9 )
10
11 from sklearn.calibration import calibration_curve

```

```

12 from scipy.stats import spearmanr, pearsonr
13 import pandas as pd
14 import matplotlib.pyplot as plt
15 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
16
17 _EPS = 1e-12
18
19 def _to_numpy(x: Any) -> np.ndarray:
20     # Torch -> numpy (without importing torch)
21     if hasattr(x, "detach") and callable(x.detach):
22         x = x.detach()
23     if hasattr(x, "cpu") and callable(x.cpu):
24         x = x.cpu()
25     if hasattr(x, "numpy") and callable(x.numpy):
26         return np.asarray(x.numpy())
27     return np.asarray(x)
28
29
30 def _ensure_2d(a: np.ndarray) -> np.ndarray:
31     a = np.asarray(a)
32     if a.ndim == 1:
33         return a.reshape(-1, 1)
34     if a.ndim != 2:
35         raise ValueError(f"Expected 2D array (or 1D), got shape {a.shape}")
36     return a
37
38
39 def _softmax(logits: np.ndarray, axis: int = 1) -> np.ndarray:
40     logits = np.asarray(logits, dtype=np.float64)
41     m = np.max(logits, axis=axis, keepdims=True)
42     exps = np.exp(logits - m)
43     return exps / np.clip(np.sum(exps, axis=axis, keepdims=True), _EPS, None)
44
45
46 def _sigmoid(z: np.ndarray) -> np.ndarray:
47     z = np.asarray(z, dtype=np.float64)
48     out = np.empty_like(z, dtype=np.float64)
49     pos = z >= 0
50     neg = ~pos
51     out[pos] = 1.0 / (1.0 + np.exp(-z[pos]))
52     ez = np.exp(z[neg])
53     out[neg] = ez / (1.0 + ez)
54     return out
55
56
57 def _labels_from_y(y: np.ndarray) -> np.ndarray:
58     y = _ensure_2d(y)
59     if y.shape[1] == 1:
60         return y.reshape(-1).astype(int)
61     return np.argmax(y, axis=1).astype(int)
62
63
64 def _onehot_from_labels(labels: np.ndarray, n_classes: int) -> np.ndarray:
65     labels = labels.astype(int)
66     oh = np.zeros((labels.shape[0], n_classes), dtype=int)
67     oh[np.arange(labels.shape[0]), labels] = 1
68     return oh
69
70
71 def _pearson_1d(a: np.ndarray, b: np.ndarray) -> float:
72     a = np.asarray(a, dtype=np.float64).reshape(-1)
73     b = np.asarray(b, dtype=np.float64).reshape(-1)
74     try:
75         r, _ = pearsonr(a, b)
76         return float(r)
77     except Exception:
78         return float("nan")
79
80
81 def _spearman_1d(a: np.ndarray, b: np.ndarray) -> float:
82     a = np.asarray(a, dtype=np.float64).reshape(-1)
83     b = np.asarray(b, dtype=np.float64).reshape(-1)

```



```

84 try:
85     res = spearmanr(a, b)
86     corr = getattr(res, "correlation", res)
87     return float(corr)
88 except Exception:
89     return float("nan")
90
91
92 def _calibration_stats_binary(
93     y_true01: np.ndarray,
94     p_pos: np.ndarray,
95     n_bins: int = 10
96 ) -> Tuple[float, List[float], List[float]]:
97     """
98     Returns:
99     - avg_abs_gap: mean(|prob_true - prob_pred|) across non-empty bins
100     - prob_pred_list
101     - prob_true_list
102     """
103     y_true01 = np.asarray(y_true01).astype(int).reshape(-1)
104     p_pos = np.asarray(p_pos, dtype=np.float64).reshape(-1)
105     p_pos = np.clip(p_pos, _EPS, 1.0 - _EPS)
106
107     # calibration_curve can fail if only one class present
108     prob_true, prob_pred = calibration_curve(
109         y_true01,
110         p_pos,
111         n_bins=n_bins,
112         strategy="quantile"
113     )
114     gap = np.abs(prob_true - prob_pred)
115     avg_abs_gap = float(np.mean(gap)) if gap.size > 0 else float("nan")
116     return avg_abs_gap, [float(x) for x in prob_pred], [float(x) for x in prob_true]
117
118
119 def binary_classification_metrics(
120     y_score: Any,
121     y: Any,
122     threshold: float = 0.5,
123     input_type: str = "probs", # "probs" or "logits"
124     pos_index: int = 1,
125     n_calib_bins: int = 10
126 ) -> Dict[str, float]:
127     """
128     Binary classification metrics.
129
130     Inputs:
131     y_score: (n,1) prob/logit for positive class OR (n,2) probs/logits for both classes.
132     y: (n,1) labels in {0,1} OR (n,2) one-hot.
133     Returns:
134     dict with:
135     acc/precision/recall/f1 at threshold,
136     cross_entropy, auc,
137     brier,
138     calib_abs_gap (quantile bins),
139     calib_curve_prob_pred, calib_curve_prob_true (lists)
140     """
141     ys = _ensure_2d(_to_numpy(y_score))
142     yy = _ensure_2d(_to_numpy(y))
143     y_true = _labels_from_y(yy)
144
145     if input_type not in ("probs", "logits"):
146         raise ValueError("input_type must be 'probs' or 'logits'")
147
148     if ys.shape[1] == 1:
149         if input_type == "probs":
150             p_pos = ys.reshape(-1).astype(np.float64)
151         else:
152             p_pos = _sigmoid(ys.reshape(-1))
153     elif ys.shape[1] == 2:
154         if pos_index not in (0, 1):
155             raise ValueError("pos_index must be 0 or 1 for binary (n,2) scores.")

```

```

156     if input_type == "probs":
157         probs2 = ys.astype(np.float64)
158     else:
159         probs2 = _softmax(ys, axis=1)
160     p_pos = probs2[:, pos_index]
161     else:
162         raise ValueError(f"Binary expects y_score with 1 or 2 columns, got {ys.shape[1]}")
163
164     p_pos = np.clip(p_pos, _EPS, 1.0 - _EPS)
165     y_pred = (p_pos >= threshold).astype(int)
166
167     tp = int(np.sum((y_true == 1) & (y_pred == 1)))
168     fp = int(np.sum((y_true == 0) & (y_pred == 1)))
169     fn = int(np.sum((y_true == 1) & (y_pred == 0)))
170     tn = int(np.sum((y_true == 0) & (y_pred == 0)))
171
172     precision = 0.0 if (tp + fp) == 0 else tp / (tp + fp)
173     recall = 0.0 if (tp + fn) == 0 else tp / (tp + fn)
174     f1 = 0.0 if (precision + recall) == 0 else 2.0 * precision * recall / (precision + recall)
175     acc = (tp + tn) / max(1, (tp + tn + fp + fn))
176
177     out: Dict[str, float] = {}
178     out[f"acc@{threshold:g}"] = float(acc)
179     out[f"precision@{threshold:g}"] = float(precision)
180     out[f"recall@{threshold:g}"] = float(recall)
181     out[f"f1@{threshold:g}"] = float(f1)
182
183     # threshold-free metrics
184     try:
185         out["cross_entropy"] = float(log_loss(y_true, p_pos, labels=[0, 1]))
186     except Exception:
187         out["cross_entropy"] = float("nan")
188
189     try:
190         out["auc"] = float(roc_auc_score(y_true, p_pos))
191     except Exception:
192         out["auc"] = float("nan")
193
194     # calibration: Brier + calibration curve gap
195     try:
196         out["brier"] = float(brier_score_loss(y_true, p_pos))
197     except Exception:
198         out["brier"] = float("nan")
199
200     try:
201         calib_gap, prob_pred, prob_true = _calibration_stats_binary(y_true, p_pos, n_bins=n_calib_bins)
202         out["calib_abs_gap"] = float(calib_gap)
203         # "calibration curves" as points: (prob_pred, prob_true)
204         out["calib_curve_prob_pred"] = prob_pred
205         out["calib_curve_prob_true"] = prob_true
206     except Exception:
207         out["calib_abs_gap"] = float("nan")
208         out["calib_curve_prob_pred"] = []
209         out["calib_curve_prob_true"] = []
210
211     return out
212
213
214 def multiclass_classification_metrics(
215     y_score: Any,
216     y: Any,
217     input_type: str = "probs",    # "probs" or "logits"
218     multi_class_auc: str = "ovr", # "ovr" or "ovo"
219     n_calib_bins: int = 10
220 ) -> Dict[str, float]:
221     """
222     Multinomial (single-label) classification metrics.
223
224     Inputs:
225         y_score: (n,C) probs or logits
226         y: (n,1) integer labels OR (n,C) one-hot
227     Returns:

```

```

228     dict with:
229         acc,
230         precision/recall/f1 (macro/micro/weighted),
231         cross_entropy,
232         auc_(macro/micro/weighted),
233         brier (multiclass),
234         calib_abs_gap_(macro/micro/weighted) computed via OVR calibration curves,
235         plus calibration curve points (per-class + micro).
236     """
237     ys = _ensure_2d(_to_numpy(y_score))
238     yy = _ensure_2d(_to_numpy(y))
239     y_true = _labels_from_y(yy)
240
241     if ys.shape[1] < 2:
242         raise ValueError(f"Multiclass expects y_score with C>=2 columns, got {ys.shape[1]}")
243
244     if input_type not in ("probs", "logits"):
245         raise ValueError("input_type must be 'probs' or 'logits'")
246
247     if multi_class_auc not in ("ovr", "ovo"):
248         raise ValueError("multi_class_auc must be 'ovr' or 'ovo'")
249
250     if input_type == "probs":
251         probs = ys.astype(np.float64)
252     else:
253         probs = _softmax(ys, axis=1)
254
255     probs = np.clip(probs, _EPS, 1.0)
256     probs = probs / np.clip(probs.sum(axis=1, keepdims=True), _EPS, None)
257
258     n_classes = probs.shape[1]
259     y_pred = np.argmax(probs, axis=1)
260     y_onehot = _onehot_from_labels(y_true, n_classes)
261
262     out: Dict[str, float] = {}
263     out["acc"] = float(accuracy_score(y_true, y_pred))
264
265     for avg in ("macro", "micro", "weighted"):
266         p, r, f, _ = precision_recall_fscore_support(
267             y_true,
268             y_pred,
269             average=avg,
270             zero_division=0
271         )
272         out[f"precision_{avg}"] = float(p)
273         out[f"recall_{avg}"] = float(r)
274         out[f"f1_{avg}"] = float(f)
275
276     # Cross-entropy
277     try:
278         out["cross_entropy"] = float(log_loss(y_true, probs, labels=list(range(n_classes))))
279     except Exception:
280         out["cross_entropy"] = float("nan")
281
282     # AUC macro/weighted (sklearn multiclass)
283     for avg in ("macro", "weighted"):
284         try:
285             out[f"auc_{avg}"] = float(
286                 roc_auc_score(y_true, probs, multi_class=multi_class_auc, average=avg)
287             )
288         except Exception:
289             out[f"auc_{avg}"] = float("nan")
290
291     # AUC micro via flattening (robust across sklearn versions)
292     try:
293         out["auc_micro"] = float(roc_auc_score(y_onehot.ravel(), probs.ravel()))
294     except Exception:
295         out["auc_micro"] = float("nan")
296
297     # Multiclass Brier score: mean over samples of sum_k (p_k - y_k)^2
298     try:
299         out["brier"] = float(np.mean(np.sum((probs - y_onehot) ** 2, axis=1)))

```

```

300 except Exception:
301     out["brier"] = float("nan")
302
303 # Calibration (OVR per class + aggregated gaps)
304 gaps_by_class: List[float] = []
305 curve_pred_by_class: List[List[float]] = []
306 curve_true_by_class: List[List[float]] = []
307
308 class_counts = np.bincount(y_true, minlength=n_classes).astype(np.float64)
309 class_weights = class_counts / max(1.0, float(np.sum(class_counts)))
310
311 for k in range(n_classes):
312     yk = y_onehot[:, k]
313     pk = probs[:, k]
314     try:
315         gap_k, prob_pred, prob_true = _calibration_stats_binary(yk, pk, n_bins=n_calib_bins)
316         gaps_by_class.append(float(gap_k))
317         curve_pred_by_class.append(prob_pred)
318         curve_true_by_class.append(prob_true)
319     except Exception:
320         gaps_by_class.append(float("nan"))
321         curve_pred_by_class.append([])
322         curve_true_by_class.append([])
323
324 out["calib_abs_gap_by_class"] = [float(x) if np.isfinite(x) else float("nan") for x in gaps_by_class]
325 out["calib_curve_prob_pred_by_class"] = curve_pred_by_class
326 out["calib_curve_prob_true_by_class"] = curve_true_by_class
327
328 # macro / weighted gaps (ignore NaNs)
329 gaps_arr = np.asarray(gaps_by_class, dtype=np.float64)
330 out["calib_abs_gap_macro"] = float(np.nanmean(gaps_arr)) if np.isfinite(gaps_arr).any() else float("nan")
331
332 try:
333     # Weighted average ignoring NaNs by re-normalizing weights over finite gaps
334     finite = np.isfinite(gaps_arr)
335     if not finite.any():
336         out["calib_abs_gap_weighted"] = float("nan")
337     else:
338         w = class_weights[finite]
339         w = w / np.clip(np.sum(w), _EPS, None)
340         out["calib_abs_gap_weighted"] = float(np.sum(w * gaps_arr[finite]))
341 except Exception:
342     out["calib_abs_gap_weighted"] = float("nan")
343
344 # micro calibration: flatten one-hot and probs, then do binary calibration
345 try:
346     gap_micro, prob_pred_micro, prob_true_micro = _calibration_stats_binary(
347         y_onehot.ravel(),
348         probs.ravel(),
349         n_bins=n_calib_bins
350     )
351     out["calib_abs_gap_micro"] = float(gap_micro)
352     out["calib_curve_prob_pred_micro"] = prob_pred_micro
353     out["calib_curve_prob_true_micro"] = prob_true_micro
354 except Exception:
355     out["calib_abs_gap_micro"] = float("nan")
356     out["calib_curve_prob_pred_micro"] = []
357     out["calib_curve_prob_true_micro"] = []
358
359 return out
360
361
362 def regression_metrics(
363     y_hat: Any,
364     y: Any
365 ) -> Dict[str, float]:
366     """
367     Regression metrics with support for multi-output regression.
368
369     Inputs:
370     y_hat: (n,d) predictions (or (n,1))
371     y: (n,d) targets (or (n,1))

```

```

372 Returns:
373     dict with overall means + per-dimension arrays:
374         mse, rmse, mae, pearson, spearman
375     """
376     yp = _ensure_2d(_to_numpy(y_hat)).astype(np.float64)
377     yt = _ensure_2d(_to_numpy(y)).astype(np.float64)
378
379     if yp.shape != yt.shape:
380         raise ValueError(f"y_hat and y must have same shape. Got {yp.shape} vs {yt.shape}")
381
382     mse_by = mean_squared_error(yt, yp, multioutput="raw_values")
383     mae_by = mean_absolute_error(yt, yp, multioutput="raw_values")
384     rmse_by = np.sqrt(np.maximum(mse_by, 0.0))
385
386     pearson_by = []
387     spearman_by = []
388     for j in range(yt.shape[1]):
389         pearson_by.append(_pearson_1d(yt[:, j], yp[:, j]))
390         spearman_by.append(_spearman_1d(yt[:, j], yp[:, j]))
391
392     mse_by = np.asarray(mse_by, dtype=np.float64)
393     rmse_by = np.asarray(rmse_by, dtype=np.float64)
394     mae_by = np.asarray(mae_by, dtype=np.float64)
395     pearson_by = np.asarray(pearson_by, dtype=np.float64)
396     spearman_by = np.asarray(spearman_by, dtype=np.float64)
397
398     out: Dict[str, float] = {}
399     out["mse"] = float(np.nanmean(mse_by))
400     out["rmse"] = float(np.nanmean(rmse_by))
401     out["mae"] = float(np.nanmean(mae_by))
402     out["pearson"] = float(np.nanmean(pearson_by))
403     out["spearman"] = float(np.nanmean(spearman_by))
404
405     out["mse_by_dim"] = [float(x) for x in mse_by]
406     out["rmse_by_dim"] = [float(x) for x in rmse_by]
407     out["mae_by_dim"] = [float(x) for x in mae_by]
408     out["pearson_by_dim"] = [float(x) if np.isfinite(x) else float("nan") for x in pearson_by]
409     out["spearman_by_dim"] = [float(x) if np.isfinite(x) else float("nan") for x in spearman_by]
410
411     return out
412
413 ## Outputsa
414 def plot_confusion_matrix(
415     y: Any,
416     y_score: Any = None,
417     y_pred: Any = None,
418     task: str = "multiclass",          # "binary" or "multiclass"
419     input_type: str = "probs",        # "probs" or "logits" (only used if y_score is given)
420     threshold: float = 0.5,          # only used for binary if y_score is given
421     pos_index: int = 1,              # only used for binary when y_score has shape (n,2)
422     normalize: Optional[str] = None, # None, "true", "pred", "all"
423     labels: Optional[List[int]] = None,
424     display_labels: Optional[List[str]] = None,
425     ax: Optional[Any] = None,
426     title: Optional[str] = None
427 ) -> Tuple[np.ndarray, Any, Any]:
428     """
429     Plot and return confusion matrix.
430
431     You can provide either:
432     - y_pred (preferred if already computed), or
433     - y_score (logits/probs) to derive y_pred
434
435     Inputs:
436     y: (n,1) labels or (n,C) one-hot
437     y_score: (n,1)/(n,2) for binary or (n,C) for multiclass
438     y_pred: (n,1) predicted labels (or (n,) is fine)
439     Returns:
440     cm: np.ndarray
441     fig, ax
442     """
443     yy = _ensure_2d(_to_numpy(y))

```

```

444 y_true = _labels_from_y(yy)
445
446 if y_pred is None:
447     if y_score is None:
448         raise ValueError("Provide either y_pred or y_score.")
449     ys = _ensure_2d(_to_numpy(y_score))
450
451     if task == "binary":
452         if input_type not in ("probs", "logits"):
453             raise ValueError("input_type must be 'probs' or 'logits'")
454
455         if ys.shape[1] == 1:
456             p_pos = ys.reshape(-1).astype(np.float64)
457             if input_type == "logits":
458                 p_pos = _sigmoid(p_pos)
459         elif ys.shape[1] == 2:
460             if pos_index not in (0, 1):
461                 raise ValueError("pos_index must be 0 or 1 for binary (n,2) scores.")
462             probs2 = ys.astype(np.float64) if input_type == "probs" else _softmax(ys, axis=1)
463             p_pos = probs2[:, pos_index]
464         else:
465             raise ValueError(f"Binary expects y_score with 1 or 2 columns, got {ys.shape[1]}")
466
467         y_pred_lbl = (p_pos >= threshold).astype(int)
468
469     elif task == "multiclass":
470         if ys.shape[1] < 2:
471             raise ValueError(f"Multiclass expects y_score with C>=2 columns, got {ys.shape[1]}")
472         probs = ys.astype(np.float64) if input_type == "probs" else _softmax(ys, axis=1)
473         y_pred_lbl = np.argmax(probs, axis=1).astype(int)
474
475     else:
476         raise ValueError("task must be 'binary' or 'multiclass'")
477 else:
478     yp = _to_numpy(y_pred)
479     if yp.ndim == 2 and yp.shape[1] == 1:
480         yp = yp.reshape(-1)
481     y_pred_lbl = yp.astype(int)
482
483 cm = confusion_matrix(
484     y_true,
485     y_pred_lbl,
486     labels=labels,
487     normalize=normalize
488 )
489
490 if ax is None:
491     fig, ax = plt.subplots()
492 else:
493     fig = ax.figure
494
495 disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=display_labels)
496 disp.plot(ax=ax, values_format=".2f" if normalize else "d", colorbar=False)
497
498 if title is None:
499     base = "Confusion Matrix"
500     if normalize:
501         base += f" (normalize={normalize})"
502     title = base
503 ax.set_title(title)
504 fig.tight_layout()
505
506 return cm, fig, ax
507
508
509 def compare_models(
510     y_hats: List[Any],
511     names: List[str],
512     y: Any,
513     task: str,          # "binary", "multiclass", "regression"
514     input_type: str = "probs",  # for classification: "probs" or "logits"
515     threshold: float = 0.5,     # for binary threshold metrics

```

```

516 pos_index: int = 1,          # binary (n,2) case
517 multi_class_auc: str = "ovr", # multiclass AUC mode
518 n_calib_bins: int = 10,
519 metrics: Optional[List[str]] = None,
520 sort_by: Optional[str] = None,
521 ascending: bool = False,
522 print_table: bool = True
523 ) -> pd.DataFrame:
524     """
525     Compare multiple models by computing metrics for each and printing a table.
526
527     Inputs:
528     y_hats: list of predictions (each can be torch/numpy). Shapes:
529         - binary: (n,1) or (n,2) probs/logits
530         - multiclass: (n,C) probs/logits
531         - regression: (n,d)
532     names: list of model names (same length as y_hats)
533     y: ground-truth (2D, but 1D tolerated)
534     metrics: optional list of column names to keep in the output table
535     sort_by: optional metric column to sort by
536     Returns:
537     pandas DataFrame
538     """
539     if len(y_hats) != len(names):
540         raise ValueError("y_hats and names must have same length")
541
542     rows = []
543     for yh, name in zip(y_hats, names):
544         if task == "binary":
545             d = binary_classification_metrics(
546                 y_score=yh,
547                 y=y,
548                 threshold=threshold,
549                 input_type=input_type,
550                 pos_index=pos_index,
551                 n_calib_bins=n_calib_bins
552             )
553             # Keep a sane default subset if caller didn't specify
554             default_cols = [
555                 f"acc@{threshold:g}",
556                 f"f1@{threshold:g}",
557                 f"precision@{threshold:g}",
558                 f"recall@{threshold:g}",
559                 "auc",
560                 "cross_entropy",
561                 "brier",
562                 "calib_abs_gap",
563             ]
564         elif task == "multiclass":
565             d = multiclass_classification_metrics(
566                 y_score=yh,
567                 y=y,
568                 input_type=input_type,
569                 multi_class_auc=multi_class_auc,
570                 n_calib_bins=n_calib_bins
571             )
572             default_cols = [
573                 "acc",
574                 "f1_macro",
575                 "f1_weighted",
576                 "precision_macro",
577                 "precision_weighted",
578                 "recall_macro",
579                 "precision_macro",
580                 "auc_macro",
581                 "auc_weighted",
582                 "cross_entropy",
583                 "brier",
584                 "calib_abs_gap_macro",
585                 "calib_abs_gap_weighted"
586             ]
587         elif task == "regression":

```

```

588     d = regression_metrics(y_hat=yh, y=y)
589     default_cols = [
590         "rmse",
591         "mae",
592         "mse",
593         "pearson",
594         "spearman",
595     ]
596     else:
597         raise ValueError("task must be 'binary', 'multiclass', or 'regression'")
598
599     row = {"model": name}
600     row.update(d)
601     rows.append(row)
602
603     df = pd.DataFrame(rows)
604
605     # If user didn't pass a metric list, keep defaults (plus model)
606     if metrics is None:
607         if len(rows) > 0:
608             if task in ("binary", "multiclass"):
609                 keep = ["model"] + [c for c in default_cols if c in df.columns]
610             else:
611                 keep = ["model"] + [c for c in default_cols if c in df.columns]
612             df = df.loc[:, keep]
613         else:
614             keep = ["model"] + [m for m in metrics if m in df.columns]
615             df = df.loc[:, keep]
616
617     if sort_by is not None and sort_by in df.columns:
618         df = df.sort_values(sort_by, ascending=bool(ascending)).reset_index(drop=True)
619     else:
620         df = df.reset_index(drop=True)
621
622     if print_table:
623         # Print nicely without scientific spam
624         with pd.option_context("display.max_columns", None, "display.width", 200):
625             print(df.to_string(index=False))
626
627     return df

```

▼ Regression

Generate data

```

1 from sklearn.datasets import make_regression
2
3 X, Y = make_regression(n_samples=10000, n_features=30, noise=1, random_state=42)

```

```

1 X, Y = torch.tensor(X, dtype=torch.float32), torch.tensor(Y[:,None], dtype=torch.float32)
2
3 (X_train, Y_train,
4  X_val, Y_val,
5  X_test, Y_test) = random_split_xy(X, Y)
6
7 train_data = TabularDataset(X_train, Y_train)
8 val_data = TabularDataset(X_val, Y_val)
9 test_data = TabularDataset(X_test, Y_test)

```

Sandbox

```

1 batch_size = 128
2 early_stop_patience = 5
3 early_stop_tol = 0
4 eval_every = 10
5 n_epochs = 1000

```



```

6 loss_fn = nn.MSELoss()
7
8 lr = 1e-2
9 weight_decay = 0.001

```

```

1 model = LinearModel(d_in=X.shape[1], d_out=Y.shape[1])
2 train_dataloader = DataLoader(train_data, batch_size=128, shuffle=True)
3 val_dataloader = DataLoader(val_data, batch_size=128, shuffle=False)
4
5 optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
6 hist = []
7
8 for epoch in tqdm(range(n_epochs)):
9     for batch in train_dataloader:
10         model.zero_grad()
11         x, y = batch
12         loss = loss_fn(model(x), y)
13         hist.append(loss.item())
14         loss.backward()
15         optimizer.step()

```

100%

1000/1000 [01:19<00:00, 14.09it/s]

Clique duas vezes (ou pressione "Enter") para editar

```
1 np.min(hist)
```

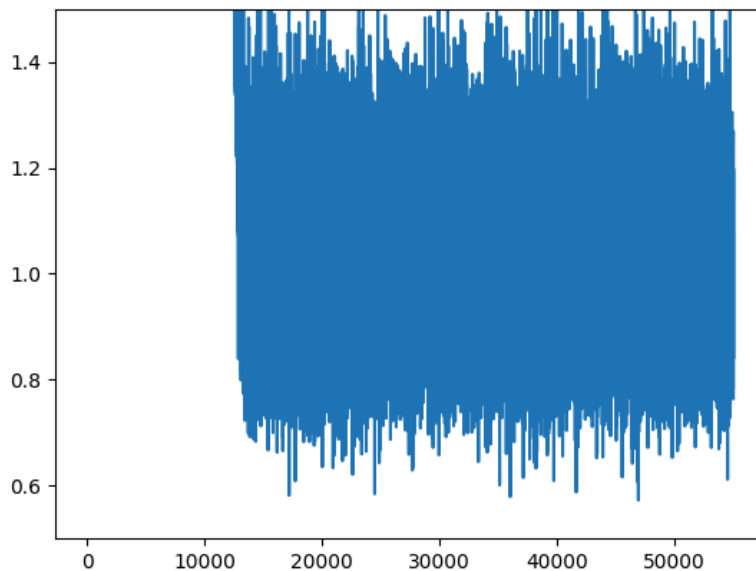
```
np.float64(0.5725356936454773)
```

```

1 plt.plot(hist)
2 plt.ylim(.5,1.5)

```

(0.5, 1.5)



```

1 preds = []
2 model.eval()
3 for batch in train_dataloader:
4     if isinstance(batch, (tuple, list)) and len(batch) == 2:
5         x, y = batch
6     else:
7         x, y = batch, None
8
9     y_hat = model(x)
10
11     preds.append(y_hat.detach().cpu())

```

```

12
13 y_hat_all = torch.cat(preds, dim=0) if len(preds) > 0 else torch.empty(0)

```

```

1 len(preds)

```

```

55

```

Training

```

1 batch_size = 128
2 early_stop_patience = 5
3 early_stop_tol = 0
4 eval_every = 100
5 n_steps = 100_000
6 loss_fn = nn.MSELoss()
7
8 lr = 1e-1
9 weight_decay = 0.0

```

```

1 #linear_reg = LinearModel(d_in=X.shape[1], d_out=Y.shape[1])
2 #linear_reg.linear.weight, linear_reg.linear.bias

```

```

1 output1 = fit_adam(model = LinearModel(d_in=X.shape[1], d_out=Y.shape[1]),
2                     train_data = train_data,
3                     val_data = val_data,
4                     batch_size = batch_size,
5                     early_stop_patience = early_stop_patience,
6                     early_stop_tol = early_stop_tol,
7                     eval_every = eval_every,
8                     lr = lr,
9                     weight_decay = weight_decay,
10                    n_steps = n_steps,
11                    loss_fn = loss_fn)

```

epoch 49: 3%

2740/100000 [00:11<06:34, 246.69step/s, bad=5, best_val=1.055, train=0.9455, val=1.066]

```

1 output2 = fit_adam(model = MLP(d_in=X.shape[1],
2                                d_hidden = [100],
3                                activation='relu',
4                                d_out=Y.shape[1]),
5                    train_data = train_data,
6                    val_data = val_data,
7                    batch_size = batch_size,
8                    early_stop_patience = early_stop_patience,
9                    early_stop_tol = early_stop_tol,
10                   eval_every = eval_every,
11                   lr = lr/100,
12                   weight_decay = weight_decay,
13                   n_steps = n_steps,
14                   loss_fn = loss_fn)

```

epoch 283: 16%

15600/100000 [01:10<05:26, 258.62step/s, bad=5, best_val=3.17, train=0.9751, val=3.26]

```

1 output3 = fit_lbfgs(model = LinearModel(d_in=X.shape[1], d_out=Y.shape[1]),
2                     train_data = train_data,
3                     val_data = val_data,
4                     loss_fn = loss_fn)
5 output3['model'] = output3['model'].float()

```

```

1 y_hats = [predict(model=output1['model'], data=test_data)[0],
2           predict(model=output2['model'], data=test_data)[0],
3           predict(model=output3['model'], data=test_data)[0]]
4
5 compare_models(y_hats = y_hats,
6               names = ['linear', 'mlp', 'lbfgs'],

```

```

7         y = Y_test,
8         task = 'regression')

```

model	rmse	mae	mse	pearson	spearman
linear	1.000982	0.800860	1.001966	0.999985	0.999978
mlp	1.859514	1.427556	3.457791	0.999949	0.999924
lbfgs	1.002087	0.800804	1.004178	0.999985	0.999979

	model	rmse	mae	mse	pearson	spearman
0	linear	1.000982	0.800860	1.001966	0.999985	0.999978
1	mlp	1.859514	1.427556	3.457791	0.999949	0.999924
2	lbfgs	1.002087	0.800804	1.004178	0.999985	0.999979

```

1 y_hats = [predict(model=output1['model'], data=val_data)[0],
2           predict(model=output2['model'], data=val_data)[0],
3           predict(model=output3['model'], data=val_data)[0]]
4
5 compare_models(y_hats = y_hats,
6               names = ['linear', 'mlp', 'lbfgs'],
7               y = Y_val,
8               task = 'regression')

```

model	rmse	mae	mse	pearson	spearman
linear	1.027771	0.800825	1.056312	0.999985	0.999977
mlp	1.966798	1.494743	3.868294	0.999947	0.999915
lbfgs	1.029260	0.802247	1.059376	0.999985	0.999977

	model	rmse	mae	mse	pearson	spearman
0	linear	1.027771	0.800825	1.056312	0.999985	0.999977
1	mlp	1.966798	1.494743	3.868294	0.999947	0.999915
2	lbfgs	1.029260	0.802247	1.059376	0.999985	0.999977

Binary Classification

```

1 from sklearn.datasets import make_classification
2 X, Y = make_classification(n_samples=10000, random_state=42)
3
4 X, Y = torch.tensor(X, dtype=torch.float32), torch.tensor(Y[:,None], dtype=torch.float32)
5
6 (X_train, Y_train,
7  X_val, Y_val,
8  X_test, Y_test) = random_split_xy(X, Y)
9
10 train_data = TabularDataset(X_train, Y_train)
11 val_data = TabularDataset(X_val, Y_val)
12 test_data = TabularDataset(X_test, Y_test)
13
14 #Y_train = Y_train.long()
15 #Y_val = Y_val.long()
16 #Y_test = Y_test.long()
17
18 X_train.shape, Y_train.shape

```

```
(torch.Size([7000, 20]), torch.Size([7000, 1]))
```

Training

```

1 batch_size = 128
2 early_stop_patience = 5
3 early_stop_tol = 0
4 eval_every = 100
5 n_steps = 100_000
6 loss_fn = nn.BCEWithLogitsLoss()
7

```

```
8 lr = 1e-1
9 weight_decay = 0.0
```

```
1 model = LinearModel(d_in=X.shape[1], d_out=Y.shape[1])
2 p = model(X_train)
3 loss_fn(p, Y_train)
```

```
tensor(0.7008, grad_fn=<BinaryCrossEntropyWithLogitsBackward0>)
```

```
1 output1 = fit_adam(model = LinearModel(d_in=X.shape[1], d_out=Y.shape[1]),
2                     train_data = train_data,
3                     val_data = val_data,
4                     batch_size = batch_size,
5                     early_stop_patience = early_stop_patience,
6                     early_stop_tol = early_stop_tol,
7                     eval_every = eval_every,
8                     lr = lr,
9                     weight_decay = weight_decay,
10                    n_steps = n_steps,
11                    loss_fn = loss_fn)
```

epoch 12: 1%

700/100000 [00:03<05:13, 316.92step/s, bad=5, best_val=0.3045, train=0.3281, val=0.3116]

```
1 output2 = fit_adam(model = MLP(d_in=X.shape[1],
2                                d_hidden = [100],
3                                activation='relu',
4                                d_out=Y.shape[1]),
5                    train_data = train_data,
6                    val_data = val_data,
7                    batch_size = batch_size,
8                    early_stop_patience = early_stop_patience,
9                    early_stop_tol = early_stop_tol,
10                   eval_every = eval_every,
11                   lr = lr/100,
12                   weight_decay = weight_decay,
13                   n_steps = n_steps,
14                   loss_fn = loss_fn)
```

epoch 45: 2%

2500/100000 [00:16<08:40, 187.38step/s, bad=5, best_val=0.2418, train=0.2088, val=0.2428]

```
1 output3 = fit_lbfgs(model = LinearModel(d_in=X.shape[1], d_out=Y.shape[1]),
2                     train_data = train_data,
3                     val_data = val_data,
4                     loss_fn = loss_fn)
5 output3['model'] = output3['model'].float()
```

```
1 y_hats = [predict(model=output1['model'], data=test_data)[0],
2            predict(model=output2['model'], data=test_data)[0],
3            predict(model=output3['model'], data=test_data)[0]]
4 for i,y_hat in enumerate(y_hats):
5     y_hats[i] = sigmoid(y_hat)
6
7 compare_models(y_hats = y_hats,
8               names = ['linear', 'mlp', 'lbfgs'],
9               y = Y_test,
10              task = 'binary')
```

model	acc@0.5	f1@0.5	precision@0.5	recall@0.5	auc	cross_entropy	brier	calib_abs_gap
linear	0.879	0.881139	0.884615	0.877691	0.939585	0.319796	0.092980	0.019171
mlp	0.927	0.929605	0.916350	0.943249	0.952268	0.273921	0.064472	0.035131
lbfgs	0.879	0.880906	0.886139	0.875734	0.942565	0.308973	0.089454	0.019383

	model	acc@0.5	f1@0.5	precision@0.5	recall@0.5	auc	cross_entropy	brier	calib_abs_gap
0	linear	0.879	0.881139	0.884615	0.877691	0.939585	0.319796	0.092980	0.019171
1	mlp	0.927	0.929605	0.916350	0.943249	0.952268	0.273921	0.064472	0.035131
2	lbfgs	0.879	0.880906	0.886139	0.875734	0.942565	0.308973	0.089454	0.019383

```
1 ((y_hats[1]>.5) == (Y_test>.5)).float().mean()
```

```
tensor(0.9270)
```

```
1 Y_test
```

```
tensor([[0.],
        [1.],
        [1.],
        ...,
        [1.],
        [1.],
        [0.]])
```

✓ Multinomial Classification

```
1 from sklearn.datasets import make_classification
2 X, Y = make_classification(n_samples=10000, n_classes=5, n_informative=6, random_state=42)
3
4 X, Y = torch.tensor(X, dtype=torch.float32), torch.tensor(Y[:,None], dtype=torch.float32)
5
6 (X_train, Y_train,
7  X_val, Y_val,
8  X_test, Y_test) = random_split_xy(X, Y)
9
10
11 Y_train = Y_train.long().squeeze()
12 Y_val = Y_val.long().squeeze()
13 Y_test = Y_test.long().squeeze()
14
15 train_data = TabularDataset(X_train, Y_train)
16 val_data = TabularDataset(X_val, Y_val)
17 test_data = TabularDataset(X_test, Y_test)
18
19
20 X_train.shape, Y_train.shape
```

```
(torch.Size([7000, 20]), torch.Size([7000]))
```

```
1 Y_train
```

```
tensor([2, 0, 4, ..., 1, 4, 2])
```

Training

```
1 batch_size = 128
2 early_stop_patience = 5
3 early_stop_tol = 0
4 eval_every = 100
5 n_steps = 100_000
6 loss_fn = nn.CrossEntropyLoss()
7
8 lr = 1e-1
9 weight_decay = 0.0
```

```
1 p
```

```
tensor([[ 0.0688, -1.4447,  0.6955,  0.2867, -1.2949],
        [ 0.1709,  0.1783,  0.4318, -0.7644,  0.5419],
        [ 0.0610, -0.4636, -1.3184, -0.3495,  0.8040],
        ...,
        [-0.0826, -1.5828,  0.5130,  0.6477, -0.1501],
        [ 0.3654,  0.3297,  0.0973, -0.5541, -0.2320],
        [ 0.1044, -0.7519,  0.5022, -0.1934, -1.4229]],
        grad_fn=<AddmmBackward0>)
```

```
1 Y_train
```

```
tensor([2, 0, 4, ..., 1, 4, 2])
```

```
1 model = LinearModel(d_in=X.shape[1], d_out=np.unique(Y).shape[0])
2 p = model(X_train)
3 loss_fn(p, Y_train)
```

```
tensor(1.7578, grad_fn=<NllLossBackward0>)
```

```
1 np.mean([-np.log(softmax(p)[i,j].item()) for i,j in enumerate(Y_train)])
```

```
np.float64(1.7577501028517915)
```

```
1 output1 = fit_adam(model = LinearModel(d_in=X.shape[1], d_out=np.unique(Y).shape[0]),
2                       train_data = train_data,
3                       val_data = val_data,
4                       batch_size = batch_size,
5                       early_stop_patience = early_stop_patience,
6                       early_stop_tol = early_stop_tol,
7                       eval_every = eval_every,
8                       lr = lr,
9                       weight_decay = weight_decay,
10                      n_steps = n_steps,
11                      loss_fn = loss_fn)
```

```
epoch 12: 1%
```

```
700/100000 [00:02<06:02, 274.05step/s, bad=5, best_val=1.147, train=1.186, val=1.162]
```

```
1 output2 = fit_adam(model = MLP(d_in=X.shape[1],
2                                d_hidden = [100],
3                                activation='relu',
4                                d_out=np.unique(Y).shape[0]),
5                      train_data = train_data,
6                      val_data = val_data,
7                      batch_size = batch_size,
8                      early_stop_patience = early_stop_patience,
9                      early_stop_tol = early_stop_tol,
10                     eval_every = eval_every,
11                     lr = lr/100,
12                     weight_decay = weight_decay,
13                     n_steps = n_steps,
14                     loss_fn = loss_fn)
```

```
epoch 90: 5%
```

```
5000/100000 [00:34<13:18, 118.91step/s, bad=5, best_val=0.5611, train=0.3503, val=0.5624]
```

```
1 output3 = fit_lbfgs(model = LinearModel(d_in=X.shape[1], d_out=np.unique(Y).shape[0]),
2                      train_data = train_data,
3                      val_data = val_data,
4                      loss_fn = loss_fn)
5 output3['model'] = output3['model'].float()
```

```
1 y_hats = [predict(model=output1['model'], data=test_data)[0],
2            predict(model=output2['model'], data=test_data)[0],
3            predict(model=output3['model'], data=test_data)[0]]
4 for i,y_hat in enumerate(y_hats):
5     y_hats[i] = sigmoid(y_hat)
6
7 compare_models(y_hats = y_hats,
8                names = ['linear', 'mlp', 'lbfgs'],
9                y = Y_test,
10               task = 'multiclass')
```

	model	acc	f1_macro	f1_weighted	precision_macro	precision_weighted	recall_macro	precision_macro	auc
	linear	0.557	0.552754	0.552469	0.554984	0.555404	0.558001	0.554984	0.
	mlp	0.807	0.807937	0.807069	0.808461	0.807472	0.807745	0.808461	0.
	lbfgs	0.569	0.565234	0.565337	0.564458	0.564900	0.569225	0.564458	0.
	model	acc	f1_macro	f1_weighted	precision_macro	precision_weighted	recall_macro	precision_macro	auc
0	linear	0.557	0.552754	0.552469	0.554984	0.555404	0.558001	0.554984	0
1	mlp	0.807	0.807937	0.807069	0.808461	0.807472	0.807745	0.808461	0
2	lbfgs	0.569	0.565234	0.565337	0.564458	0.564900	0.569225	0.564458	0

Matrix completion

```

1 def _rng(seed: int) -> np.random.Generator:
2     return np.random.default_rng(int(seed))
3
4 def make_matrix_completion_data(
5     n_users: int = 100,
6     n_items: int = 80,
7     rank: int = 5,
8     n_obs: int = 20000,
9     noise_std: float = 0.1,
10    seed: int = 0,
11    implicit: bool = False,
12    implicit_thresh: float = 0.0,
13    return_full_matrix: bool = False
14 ) -> Tuple[np.ndarray, np.ndarray, np.ndarray, Dict[str, np.ndarray]]:
15     """
16     Generate a low-rank matrix  $R = U V^T + \text{noise}$  and sample observed entries.
17
18     Returns:
19         user_id: (n_obs, 1) int64
20         item_id: (n_obs, 1) int64
21         y: (n_obs, 1) float32 (ratings) OR int64 (implicit labels) if implicit=True
22         info: dict containing U, V, and (optional) full_matrix
23
24     Notes:
25         - Observations are sampled uniformly over user-item pairs (with replacement).
26         - If you want unique pairs, you can post-process, but for training MF this is fine.
27     """
28     rng = _rng(seed)
29
30     U = rng.standard_normal((n_users, rank)).astype(np.float32)
31     V = rng.standard_normal((n_items, rank)).astype(np.float32)
32
33     user_id = rng.integers(0, n_users, size=(n_obs,)), dtype=np.int64
34     item_id = rng.integers(0, n_items, size=(n_obs,)), dtype=np.int64
35
36     ratings = (U[user_id] * V[item_id]).sum(axis=1)
37     ratings = ratings + noise_std * rng.standard_normal((n_obs,)).astype(np.float32)
38
39     if implicit:
40         y = (ratings > implicit_thresh).astype(np.int64).reshape(-1, 1)
41     else:
42         y = ratings.astype(np.float32).reshape(-1, 1)
43
44     info: Dict[str, np.ndarray] = {"U": U, "V": V}
45
46     if return_full_matrix:
47         full = (U @ V.T).astype(np.float32)
48         info["full_matrix"] = full
49
50     return user_id.reshape(-1, 1), item_id.reshape(-1, 1), y, info

```

```

1 I, J, Y, info = make_matrix_completion_data(noise_std=0)
2 X = torch.tensor(np.hstack((I, J)))

```

```

3 Y = torch.tensor(Y)
4
5 (X_train, Y_train,
6  X_val, Y_val,
7  X_test, Y_test) = random_split_xy(X, Y)
8
9 X_train = X_train.long().squeeze()
10 X_val = X_val.long().squeeze()
11 X_test = X_test.long().squeeze()
12
13 train_data = TabularDataset(X_train, Y_train)
14 val_data = TabularDataset(X_val, Y_val)
15 test_data = TabularDataset(X_test, Y_test)
16
17 ((Y_train-(torch.tensor(info['U'])[X_train[:,0]]*torch.tensor(info['V'])[X_train[:,1]]).sum(1)[:None]))

```

tensor(1.4303e-14)

```

1 batch_size = 128
2 early_stop_patience = 5
3 early_stop_tol = 0
4 eval_every = 100
5 n_steps = 100_000
6 loss_fn = nn.MSELoss()
7
8 lr = 1e-2
9 weight_decay = 1e-5

```

```

1 output1 = fit_adam(model = MF(100, 80, 5),
2                     train_data = train_data,
3                     val_data = val_data,
4                     batch_size = batch_size,
5                     early_stop_patience = early_stop_patience,
6                     early_stop_tol = early_stop_tol,
7                     eval_every = eval_every,
8                     lr = lr,
9                     weight_decay = weight_decay,
10                    n_steps = n_steps,
11                    loss_fn = loss_fn)

```

epoch 20: 2% 2300/100000 [00:14<09:11, 177.26step/s, bad=5, best_val=1.251e-06, train=8.665e-07, val=1.288e-06]

```

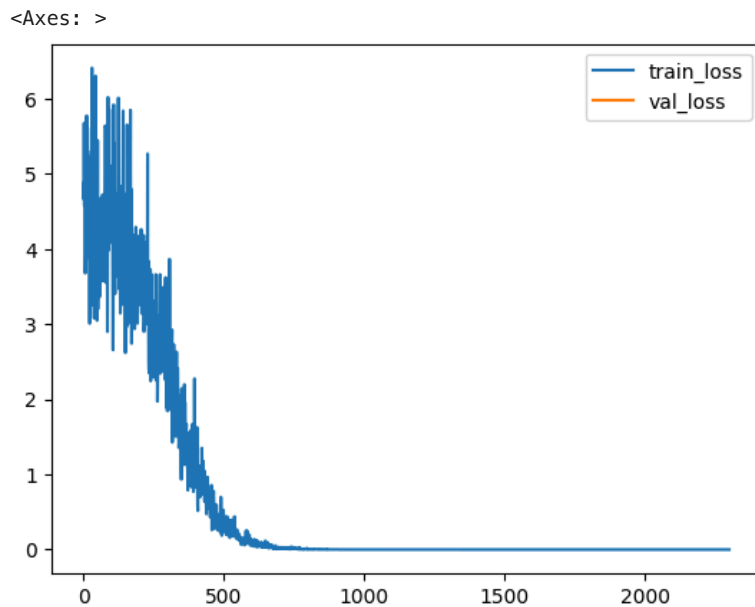
1 output3 = fit_lbfgs(model = MF(100, 80, 5),
2                     train_data = train_data,
3                     val_data = val_data,
4                     loss_fn = loss_fn)
5 output3['model'] = output3['model'].float()

```

```
1 output3['val_loss']
```

3.0079226733960233e-09

```
1 pd.DataFrame(output1['history']).loc[:, ['train_loss', 'val_loss']].plot()
```

✓ LSTM

✓ Many-to-one

```

1 import numpy as np
2 import torch
3 from torch import nn
4 from torch.utils.data import Dataset, DataLoader
5
6 # -----
7 # Synthetic multivariate time series ( $X_t$  in  $\mathbb{R}^{F_x}$ ,  $Y_t$  in  $\mathbb{R}^{F_y}$ )
8 # -----
9 def make_ts(T=7000, Fx=5, Fy=3, noise_x=0.2, noise_y=0.1, seed=0):
10     rng = np.random.default_rng(seed)
11
12     # Stable-ish X dynamics:  $X_t = A X_{t-1} + \text{eps}$ 
13     A = rng.normal(0, 0.25, size=(Fx, Fx)).astype(np.float32)
14     A *= 0.85 / (np.linalg.norm(A, ord=2) + 1e-6)
15
16     X = np.zeros((T, Fx), dtype=np.float32)
17     eps = rng.normal(0.0, noise_x, size=(T, Fx)).astype(np.float32)
18     for t in range(1, T):
19         X[t] = (A @ X[t - 1]) + eps[t]
20
21     # Y depends on current/past X and past Y (ARX/NARX-ish)
22     B0 = rng.normal(0, 0.8, size=(Fy, Fx)).astype(np.float32) #  $X_t \rightarrow Y_t$ 
23     B1 = rng.normal(0, 0.4, size=(Fy, Fx)).astype(np.float32) #  $X_{t-1} \rightarrow Y_t$ 
24     C1 = rng.normal(0, 0.3, size=(Fy, Fy)).astype(np.float32) #  $Y_{t-1} \rightarrow Y_t$ 
25     C1 *= 0.75 / (np.linalg.norm(C1, ord=2) + 1e-6)
26
27     Y = np.zeros((T, Fy), dtype=np.float32)
28     ny = rng.normal(0.0, noise_y, size=(T, Fy)).astype(np.float32)
29     for t in range(1, T):
30         lin = (B0 @ X[t]) + (B1 @ X[t - 1]) + (C1 @ Y[t - 1])
31         Y[t] = lin + 0.2 * np.tanh(lin) + ny[t]
32
33     return X, Y
34
35 # -----
36 # Windowed dataset:
37 #   input =  $X_{t-L+1:t}$  and (optionally) shifted Y history  $Y_{t-L+1:t-1}$ 
38 #   target =  $Y_t$ 
39 #

```

```

40 # include_past_y=True does NOT leak: we shift Y right and pad the first element.
41 # -----
42 class WindowDataset(Dataset):
43     def __init__(self, X, Y, seq_len=32, include_past_y=False):
44         assert len(X) == len(Y)
45         self.X = torch.tensor(X, dtype=torch.float32) # (T, Fx)
46         self.Y = torch.tensor(Y, dtype=torch.float32) # (T, Fy)
47         self.seq_len = seq_len
48         self.include_past_y = include_past_y
49
50     def __len__(self):
51         return len(self.X) - self.seq_len + 1
52
53     def __getitem__(self, idx):
54         x_seq = self.X[idx:idx + self.seq_len] # (S, Fx)
55         y_t = self.Y[idx + self.seq_len - 1] # (Fy,)
56
57         if self.include_past_y:
58             y_hist = self.Y[idx:idx + self.seq_len - 1] # (S-1, Fy) past only
59             pad = torch.zeros(1, y_hist.shape[1], dtype=y_hist.dtype)
60             y_seq = torch.cat([pad, y_hist], dim=0) # (S, Fy) shifted right
61             inp = torch.cat([x_seq, y_seq], dim=-1) # (S, Fx+Fy)
62         else:
63             inp = x_seq # (S, Fx)
64
65         return inp, y_t # (S, Fin), (Fy,)
66
67 # -----
68 # LSTM regressor: (B, S, Fin) -> (B, Fy)
69 # -----
70 class LSTMRegressor(nn.Module):
71     def __init__(self, in_features, out_features, hidden=64, num_layers=1, dropout=0.0):
72         super().__init__()
73         self.lstm = nn.LSTM(
74             input_size=in_features,
75             hidden_size=hidden,
76             num_layers=num_layers,
77             dropout=dropout if num_layers > 1 else 0.0,
78             batch_first=True
79         )
80         self.head = nn.Linear(hidden, out_features)
81
82     def forward(self, x):
83         out, _ = self.lstm(x) # (B, S, H)
84         last = out[:, -1, :] # (B, H)
85         return self.head(last) # (B, Fy)

```

```

1 T, Fx, Fy = 7000, 5, 3
2 seq_len = 32
3 include_past_y = False
4 epochs = 10
5 batch_size = 128
6 lr = 1e-3
7 seed = 0
8 early_stop_patience = 5
9 early_stop_tol = 0
10 eval_every = 100
11 n_steps = 100_000
12 weight_decay = 1e-5
13 loss_fn = nn.MSELoss()
14 torch.manual_seed(seed)
15 np.random.seed(seed)
16 in_features = Fx + (Fy if include_past_y else 0)
17 X, Y = make_ts(T=T, Fx=Fx, Fy=Fy, seed=seed)
18 X.shape, Y.shape

```

```
((7000, 5), (7000, 3))
```

```

1 # time split (no leakage across time)
2 split = int(0.8 * T)
3 X_tr, Y_tr = X[:split], Y[:split]

```

```
4 # keep alignment so first val window has enough history
5 X_va, Y_va = X[split - (seq_len - 1):], Y[split - (seq_len - 1):]
```

```
1 x,y=train_ds[1]
2 x.shape,y.shape
```

```
(torch.Size([32, 8]), torch.Size([3]))
```

```
1 train_data = WindowDataset(X_tr, Y_tr, seq_len=seq_len, include_past_y=include_past_y)
2 val_data = WindowDataset(X_va, Y_va, seq_len=seq_len, include_past_y=include_past_y)
3
4 model = LSTMRegressor(in_features=in_features, out_features=Fy, hidden=64)
5
6 output1 = fit_adam(model = model,
7                     train_data = train_data,
8                     val_data = val_data,
9                     batch_size = batch_size,
10                    early_stop_patience = early_stop_patience,
11                    early_stop_tol = early_stop_tol,
12                    eval_every = eval_every,
13                    lr = lr,
14                    weight_decay = weight_decay,
15                    n_steps = n_steps,
16                    loss_fn = loss_fn)
```

epoch 47: 2%

2100/100000 [01:13<51:38, 31.60step/s, bad=5, best_val=0.017, train=0.0171, val=0.01723]

```
1 output1['model'](torch.tensor(X_va)[-seq_len:][None,:])
```

```
tensor([[ 0.1394,  2.6334, -0.3220]], grad_fn=<AddmmBackward0>)
```

```
1 out, (h_n, c_n) = output1['model'].lstm(torch.tensor(X_va)[-seq_len:][None,:])
2 out.shape, h_n.shape, c_n.shape
```

```
(torch.Size([1, 32, 64]), torch.Size([1, 1, 64]), torch.Size([1, 1, 64]))
```

✓ Many-to-Many

```
1 class WindowDatasetManyToMany(torch.utils.data.Dataset):
2     def __init__(self, X, Y, seq_len=32, include_past_y=False):
3         assert len(X) == len(Y)
4         self.X = torch.tensor(X, dtype=torch.float32) # (T, Fx)
5         self.Y = torch.tensor(Y, dtype=torch.float32) # (T, Fy)
6         self.seq_len = seq_len
7         self.include_past_y = include_past_y
8
9     def __len__(self):
10         return len(self.X) - self.seq_len + 1
11
12     def __getitem__(self, idx):
13         x_seq = self.X[idx:idx + self.seq_len] # (S, Fx)
14         y_seq = self.Y[idx:idx + self.seq_len] # (S, Fy) targets for each step
15
16         if self.include_past_y:
17             # input gets shifted Y so it never sees the same-step label
18             y_hist = self.Y[idx:idx + self.seq_len - 1] # (S-1, Fy)
19             pad = torch.zeros(1, y_hist.shape[1], dtype=y_hist.dtype)
20             y_in = torch.cat([pad, y_hist], dim=0) # (S, Fy)
21             inp = torch.cat([x_seq, y_in], dim=-1) # (S, Fx+Fy)
22         else:
23             inp = x_seq # (S, Fx)
24
25         return inp, y_seq
26
27 class LSTMManyToMany(nn.Module):
28     def __init__(self, in_features, out_features, hidden=64, num_layers=1, dropout=0.0):
29         super().__init__()
```

```

30     self.lstm = nn.LSTM(
31         input_size=in_features,
32         hidden_size=hidden,
33         num_layers=num_layers,
34         dropout=dropout if num_layers > 1 else 0.0,
35         batch_first=True
36     )
37     self.head = nn.Linear(hidden, out_features)
38
39     def forward(self, x):
40         out, _ = self.lstm(x)      # (B, S, H)
41         return self.head(out)     # (B, S, Fy)
42

```

```

1 train_data = WindowDatasetManyToMany(X_tr, Y_tr, seq_len=seq_len, include_past_y=include_past_y)
2 val_data = WindowDatasetManyToMany(X_va, Y_va, seq_len=seq_len, include_past_y=include_past_y)
3
4 model = LSTMManyToMany(in_features=in_features, out_features=Fy, hidden=64)
5
6 output1 = fit_adam(model = model,
7                     train_data = train_data,
8                     val_data = val_data,
9                     batch_size = batch_size,
10                    early_stop_patience = early_stop_patience,
11                    early_stop_tol = early_stop_tol,
12                    eval_every = eval_every,
13                    lr = lr,
14                    weight_decay = weight_decay,
15                    n_steps = n_steps,
16                    loss_fn = loss_fn)

```

epoch 84: 4%

3700/100000 [02:20<49:33, 32.38step/s, bad=5, best_val=0.03112, train=0.03032, val=0.03113]

```
1 output1['model'](torch.tensor(X_va)[-seq_len:][None,:])[0][-1]
```

```
tensor([ 0.1676,  2.6193, -0.3453], grad_fn=<SelectBackward0>)
```

```
1 Comece a programar ou gere código com IA.
```

```
1 Comece a programar ou gere código com IA.
```

```
tensor([[ -2.4063, -0.3422,  0.0459],
        [ 0.9373,  0.8944,  2.2563]])
```

```

1 class LSTMCellScratch(nn.Module):
2
3     #Single-layer LSTM cell implemented with explicit gate math.
4     #Matches PyTorch LSTM gate order: (i, f, g, o).
5
6     def __init__(self, input_size: int, hidden_size: int, scale: float):
7         super().__init__()
8         self.input_size = input_size
9         self.hidden_size = hidden_size
10        self.scale = scale
11        self.Wi = nn.Parameter(torch.normal(mean=torch.zeros(4*hidden_size, input_size), std=scale))
12        self.Wh = nn.Parameter(torch.normal(mean=torch.zeros(4*hidden_size, hidden_size), std=scale))
13        self.b = nn.Parameter(torch.normal(mean=torch.zeros(4*hidden_size, 1), std=scale))
14
15    def forward(self, x_t: torch.Tensor, state):
16
17        #x_t: (B, I)
18        #state: (h_t, c_t), each (B, H)
19
20        h_t, c_t = state
21        i, f, g, o = (x_t@self.Wi.T + h_t@self.Wh.T + self.b.T).chunk(4, dim=-1)
22        i, f, g, o = torch.sigmoid(i), torch.sigmoid(f), torch.tanh(g), torch.sigmoid(o)
23        c_t = f * c_t + i * g
24        h_t = o * torch.tanh(c_t)
25

```

```

26     return (h_t, c_t)
27
28 class LSTMScratch(nn.Module):
29     #Processes a full sequence with the scratch LSTM cell.
30
31     def __init__(self, input_size: int, hidden_size: int, scale: float):
32         super().__init__()
33         self.input_size = input_size
34         self.hidden_size = hidden_size
35         self.scale = scale
36         self.cell = LSTMCellScratch(input_size, hidden_size, scale)
37
38     def forward(self, x: torch.Tensor, state=None):
39
40         #x: (B, S, I)
41         #state: optional (h0, c0) each (B, H)
42
43         #returns:
44         # out: (B, S, H)
45         # (hT, cT): final state
46
47         B, S, _ = x.shape
48         if state is None:
49             h_t = x.new_zeros(B, self.hidden_size)
50             c_t = x.new_zeros(B, self.hidden_size)
51         else:
52             h_t, c_t = state
53
54         out = []
55         for t in range(S):
56             h_t, c_t = self.cell(x[:,t,:], (h_t, c_t))
57             out.append(h_t)
58
59         out = torch.stack(out, dim=1)
60
61         return out, (h_t, c_t)
62
63 class LSTMRegressorScratch(nn.Module):
64     def __init__(self, in_features: int, out_features: int, scale: float = 1e-3, hidden: int = 64):
65         super().__init__()
66         self.input_size = in_features
67         self.output_size = out_features
68         self.hidden_size = hidden
69         self.scale = scale
70         self.lstm = LSTMScratch(self.input_size, self.hidden_size, self.scale)
71         self.linear = nn.Linear(self.hidden_size, self.output_size)
72
73     def forward(self, x: torch.Tensor):
74         _, state = self.lstm(x)
75         return self.linear(state[0])
76
77 """many-to-many
78 def forward(self, x: torch.Tensor):
79     out, _ = self.lstm(x)          # (B, S, H)
80     y_hat = self.linear(out)       # (B, S, out_features) (Linear applies on last dim)
81     return y_hat
82 """
83

```

1 Comece a programar ou [gere código](#) com IA.

1 Comece a programar ou [gere código](#) com IA.

1 Comece a programar ou [gere código](#) com IA.

1 Comece a programar ou [gere código](#) com IA.

1 Comece a programar ou [gere código](#) com IA.

```

1 """
2 import math
3 import torch
4 import torch.nn as nn
5
6
7 class LSTMCellScratch(nn.Module):
8
9     #Single-layer LSTM cell implemented with explicit gate math.
10    #Matches PyTorch LSTM gate order: (i, f, g, o).
11
12    def __init__(self, input_size: int, hidden_size: int, bias: bool = True):
13        super().__init__()
14        self.input_size = input_size
15        self.hidden_size = hidden_size
16
17        # W_ih: (4H, I), W_hh: (4H, H)
18        self.W_ih = nn.Parameter(torch.empty(4 * hidden_size, input_size))
19        self.W_hh = nn.Parameter(torch.empty(4 * hidden_size, hidden_size))
20
21        if bias:
22            self.b_ih = nn.Parameter(torch.empty(4 * hidden_size))
23            self.b_hh = nn.Parameter(torch.empty(4 * hidden_size))
24        else:
25            self.register_parameter("b_ih", None)
26            self.register_parameter("b_hh", None)
27
28        self.reset_parameters()
29
30    def reset_parameters(self):
31        # Similar spirit to PyTorch: uniform in [-k, k], k = 1/sqrt(H)
32        k = 1.0 / math.sqrt(self.hidden_size)
33        for p in self.parameters():
34            nn.init.uniform_(p, -k, k)
35
36    def forward(self, x_t: torch.Tensor, state):
37
38        #x_t: (B, I)
39        #state: (h_t, c_t), each (B, H)
40
41        h_t, c_t = state
42
43        gates = x_t @ self.W_ih.T + h_t @ self.W_hh.T
44        if self.b_ih is not None:
45            gates = gates + self.b_ih + self.b_hh
46
47        i, f, g, o = gates.chunk(4, dim=-1)
48        i = torch.sigmoid(i)
49        f = torch.sigmoid(f)
50        g = torch.tanh(g)
51        o = torch.sigmoid(o)
52
53        c_next = f * c_t + i * g
54        h_next = o * torch.tanh(c_next)
55        return h_next, c_next
56
57
58 class LSTMScratch(nn.Module):
59    #Processes a full sequence with the scratch LSTM cell.
60
61    def __init__(self, input_size: int, hidden_size: int, batch_first: bool = True):
62        super().__init__()
63        self.input_size = input_size
64        self.hidden_size = hidden_size
65        self.batch_first = batch_first
66        self.cell = LSTMCellScratch(input_size, hidden_size)
67
68    def forward(self, x: torch.Tensor, state=None):
69
70        #x: (B, S, I) if batch_first else (S, B, I)
71        #state: optional (h0, c0) each (B, H)

```

```

72
73     #returns:
74     # out: (B, S, H) if batch_first else (S, B, H)
75     # (hT, cT): final state
76
77     if not self.batch_first:
78         x = x.transpose(0, 1) # -> (B, S, I)
79
80     B, S, _ = x.shape
81     if state is None:
82         h_t = x.new_zeros(B, self.hidden_size)
83         c_t = x.new_zeros(B, self.hidden_size)
84     else:
85         h_t, c_t = state
86
87     outputs = []
88     for t in range(S):
89         h_t, c_t = self.cell(x[:, t, :], (h_t, c_t))
90         outputs.append(h_t)
91
92     out = torch.stack(outputs, dim=1) # (B, S, H)
93
94     if not self.batch_first:
95         out = out.transpose(0, 1) # -> (S, B, H)
96
97     return out, (h_t, c_t)
98
99
100 class LSTMRegressorScratch(nn.Module):
101     def __init__(self, in_features: int, out_features: int, hidden: int = 64):
102         super().__init__()
103         self.lstm = LSTMScratch(input_size=in_features, hidden_size=hidden, batch_first=True)
104         self.head = nn.Linear(hidden, out_features)
105
106     def forward(self, x: torch.Tensor):
107         out, _ = self.lstm(x) # out: (B, S, H)
108         last = out[:, -1, :] # (B, H)
109         return self.head(last) # (B, out_features)
110 """

```

```

1 class MLP(nn.Module):
2     def __init__(self, d_model: int = 128):
3         super().__init__()
4         self.d_model = d_model
5         self.mlp = nn.Sequential(
6             nn.Linear(d_model, 4*d_model),
7             nn.GELU(),
8             nn.Linear(4*d_model, d_model)
9         )
10
11     def forward(self, x):
12         return self.mlp(x)
13
14 class TransformerBlock(nn.Module):
15     def __init__(self, d_model: int = 128, n_heads: int = 4):
16         super().__init__()
17         self.d_model = d_model
18         self.n_heads = n_heads
19
20         self.attn = nn.MultiheadAttention(self.d_model, self.n_heads, batch_first = True)
21         self.ff = MLP(self.d_model)
22
23     def forward(self, x: torch.Tensor):
24         z, _ = self.attn(x, x, x)
25         z = self.ff(z) + x
26         return z

```

```

1 import torch
2 import torch.nn as nn
3
4 class MLP(nn.Module):

```

```

5  def __init__(self, d_model: int = 128, d_ff: int | None = None, dropout: float = 0.0):
6      super().__init__()
7      d_ff = d_ff or (4 * d_model)
8      self.net = nn.Sequential(
9          nn.Linear(d_model, d_ff),
10         nn.GELU(),
11         nn.Dropout(dropout),
12         nn.Linear(d_ff, d_model),
13     )
14
15  def forward(self, x: torch.Tensor) -> torch.Tensor:
16      return self.net(x)
17
18  class TransformerBlock(nn.Module):
19  def __init__(self, d_model: int = 128, n_heads: int = 4, dropout: float = 0.0):
20      super().__init__()
21      self.ln1 = nn.LayerNorm(d_model)
22      self.ln2 = nn.LayerNorm(d_model)
23      self.attn = nn.MultiheadAttention(d_model, n_heads, batch_first=True, dropout=dropout)
24      self.ff = MLP(d_model, dropout=dropout)
25      self.drop = nn.Dropout(dropout)
26
27  def forward(
28      self,
29      x: torch.Tensor,
30      attn_mask: torch.Tensor | None = None,
31      key_padding_mask: torch.Tensor | None = None,
32  ) -> torch.Tensor:
33      h = self.ln1(x)
34      attn_out, _ = self.attn(
35          h, h, h,
36          attn_mask=attn_mask,
37          key_padding_mask=key_padding_mask,
38          need_weights=False,
39      )
40      x = x + self.drop(attn_out)
41
42      h = self.ln2(x)
43      x = x + self.drop(self.ff(h))
44      return x
45
46  class Transformer(nn.Module):
47  def __init__(
48      self,
49      n_vocab: int,
50      len_seq: int,
51      d_model: int = 128,
52      n_heads: int = 4,
53      n_layers: int = 2,
54      dropout: float = 0.0,
55  ):
56      super().__init__()
57      self.len_seq = len_seq
58      self.emb = nn.Embedding(n_vocab, d_model)
59      self.pos = nn.Embedding(len_seq, d_model)
60      self.drop = nn.Dropout(dropout)
61      self.blocks = nn.ModuleList([
62          TransformerBlock(d_model, n_heads, dropout) for _ in range(n_layers)
63      ])
64      self.ln_f = nn.LayerNorm(d_model)
65
66  def forward(
67      self,
68      x: torch.Tensor, # (B, S)
69      attn_mask: torch.Tensor | None = None,
70      key_padding_mask: torch.Tensor | None = None,
71  ) -> torch.Tensor:
72      B, S = x.shape
73      if S > self.len_seq:
74          raise ValueError(f"Sequence length S={S} exceeds len_seq={self.len_seq}")
75
76      pos_ids = torch.arange(S, device=x.device) # (S,)

```



```

77     h = self.emb(x) + self.pos(pos_ids)[None, :, :] # (B, S, D)
78     h = self.drop(h)
79
80     for blk in self.blocks:
81         h = blk(h, attn_mask=attn_mask, key_padding_mask=key_padding_mask)
82
83     return self.ln_f(h)
84

```

1 128/4

32.0

1 Comece a programar ou [gere código](#) com IA.1 Comece a programar ou [gere código](#) com IA.

```

import math
from typing import Optional

import torch
import torch.nn as nn

class LearnedPositionalEncoding(nn.Module):
    def __init__(self, d_model: int, max_len: int = 4096, dropout: float = 0.0):
        super().__init__()
        self.max_len = max_len
        self.pos_emb = nn.Embedding(max_len, d_model)
        self.drop = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor, pos_offset: int = 0) -> torch.Tensor:
        # x: (B, S, D)
        b, s, d = x.shape
        if s + pos_offset > self.max_len:
            raise ValueError(f"Sequence length {s} (+ offset {pos_offset}) exceeds max_len={self.max_len}.")
        pos = torch.arange(pos_offset, pos_offset + s, device=x.device) # (S,)
        return self.drop(x + self.pos_emb(pos).unsqueeze(0)) # (B, S, D)

class TransformerEncoderLayerSimple(nn.Module):
    def __init__(
        self,
        d_model: int,
        n_heads: int,
        d_ff: int = 2048,
        dropout: float = 0.1,
        norm_first: bool = True
    ):
        super().__init__()
        self.norm_first = norm_first

        self.attn = nn.MultiheadAttention(d_model, n_heads, dropout=dropout, batch_first=True)
        self.ff = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model),
            nn.Dropout(dropout),
        )

```

```

        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.drop = nn.Dropout(dropout)

    def forward(
        self,
        x: torch.Tensor,
        key_padding_mask: Optional[torch.Tensor] = None,
        attn_mask: Optional[torch.Tensor] = None
    ) -> torch.Tensor:
        # x: (B, S, D)
        # key_padding_mask: (B, S) bool, True = padding (ignored)
        if self.norm_first:
            y = self.norm1(x)
            y, _ = self.attn(
                y, y, y,
                key_padding_mask=key_padding_mask,
                attn_mask=attn_mask,
                need_weights=False
            )
            x = x + self.drop(y)

            y = self.norm2(x)
            x = x + self.ff(y)
            return x
        else:
            y, _ = self.attn(
                x, x, x,
                key_padding_mask=key_padding_mask,
                attn_mask=attn_mask,
                need_weights=False
            )
            x = self.norm1(x + self.drop(y))
            x = self.norm2(x + self.ff(x))
            return x

class TransformerEncoderSimple(nn.Module):
    def __init__(
        self,
        d_model: int,
        n_heads: int,
        n_layers: int,
        d_ff: int = 2048,
        dropout: float = 0.1,
        norm_first: bool = True
    ):
        super().__init__()
        self.layers = nn.ModuleList([
            TransformerEncoderLayerSimple(
                d_model=d_model,
                n_heads=n_heads,
                d_ff=d_ff,
                dropout=dropout,
                norm_first=norm_first
            )
            for _ in range(n_layers)
        ])
        self.final_norm = nn.LayerNorm(d_model) if norm_first else nn.Identity()

    def forward(

```

```

        self,
        x: torch.Tensor,
        key_padding_mask: Optional[torch.Tensor] = None,
        attn_mask: Optional[torch.Tensor] = None
    ) -> torch.Tensor:
        for layer in self.layers:
            x = layer(x, key_padding_mask=key_padding_mask, attn_mask=attn_mask)
        return self.final_norm(x)

class TransformerEncoderModel(nn.Module):
    """
    - token ids: set vocab_size, pass input_ids (B, S) long
    - continuous: set in_features, pass x (B, S, F) float
    """
    def __init__(
        self,
        d_model: int = 256,
        n_heads: int = 8,
        n_layers: int = 4,
        d_ff: int = 1024,
        dropout: float = 0.1,
        max_len: int = 4096,
        vocab_size: Optional[int] = None,
        in_features: Optional[int] = None
    ):
        super().__init__()
        if (vocab_size is None) == (in_features is None):
            raise ValueError("Set exactly one of vocab_size (token ids) or in_features (continuous inputs).")

        self.is_token_input = vocab_size is not None
        if self.is_token_input:
            self.embed = nn.Embedding(vocab_size, d_model)
        else:
            self.embed = nn.Linear(in_features, d_model)

        self.pos = LearnedPositionalEncoding(d_model, max_len=max_len, dropout=dropout)
        self.enc = TransformerEncoderSimple(
            d_model=d_model,
            n_heads=n_heads,
            n_layers=n_layers,
            d_ff=d_ff,
            dropout=dropout,
            norm_first=True
        )

    def forward(
        self,
        x: torch.Tensor,
        key_padding_mask: Optional[torch.Tensor] = None,
        pos_offset: int = 0
    ) -> torch.Tensor:
        h = self.embed(x)                # (B, S, D)
        h = self.pos(h, pos_offset)      # trainable positions
        h = self.enc(h, key_padding_mask=key_padding_mask)
        return h                        # (B, S, D)

# --- tiny usage ---

# Token ids

```

```
B, S, V = 2, 8, 1000
m = TransformerEncoderModel(vocab_size=V, d_model=128, n_heads=4, n_layers=2, d_ff=256, max_len=512)
input_ids = torch.randint(0, V, (B, S))
pad_mask = torch.zeros(B, S, dtype=torch.bool)
out = m(input_ids, key_padding_mask=pad_mask) # (B, S, 128)

# Continuous features
B, S, F = 2, 12, 16
m2 = TransformerEncoderModel(in_features=F, d_model=128, n_heads=4, n_layers=2, d_ff=256, max_len=512)
x = torch.randn(B, S, F)
out2 = m2(x) # (B, S, 128)
```